

VISUAL ARCHITECTURE GUIDE

Rusty Pomodoro

A Pomodoro timer where Rust owns the rules and the OS owns the pixels



Fourteen files. Two complete user interfaces. One timer state machine that knows nothing about windows, menus, or graphics.

This document explains how the code fits together, using diagrams instead of prose wherever a diagram can carry the idea. Every file, field, and control-flow path described here is taken from the source tree in `src/`.

Read this part	You will understand
1–3	The three-layer split, the module map, and which code compiles on which platform
4–6	The timer state machine, the Pomodoro cycle, and the monotonic clock
7–8	The intent-flag pattern that lets the core stay platform-free
9–10	Settings and statistics storage, including the fixed-size cache
11–13	The AppKit shell, the egui shell, and how input reaches the timer
14–16	Themes, the release build, and what the test suite actually proves

Contents

1	The one idea that explains everything	5
1.1	The three layers	5
1.2	The mechanism: five boolean fields	5
1.3	What one second of running looks like	6
2	The module map	7
2.1	The core: four modules and two files on disk	7
2.2	The shells: two of them, plus their helpers	8
2.3	Line counts	8
2.4	Two shells, side by side	9
3	One binary, two personalities: the cfg switch	10
3.1	The matrix this produces	10
3.2	What each module is gated on	10
3.3	Why a Cargo feature rather than a runtime check	11
4	The timer state machine	12
4.1	Two orthogonal dimensions	12
4.2	Time is not stored; it is derived	12
4.3	The transition table	13
4.4	advance: where the Pomodoro rules live	13
5	The Pomodoro cycle, end to end	15
5.1	Normal case: a 4-session cycle	15
5.2	The three switches that reshape the cycle	15
5.3	Auto-start, and who decides it	15
5.4	Overtime: the display flips sign	16
6	The clock, and the display it feeds	17
6.1	Two clocks behind one type name	17
6.2	Rounding: why a fresh timer shows 25:00, not 24:59	17
6.3	How each shell spends that delay	19
7	The intent-flag pattern, in full	20
7.1	The five flags, their producers, their consumers	20
7.2	Why <code>mem::take</code> and not <code>if</code>	20
7.3	The one flag that is not a flag	20
7.4	Sleep and wake	21

8	tick(): the one function that runs every second	22
8.1	The four decisions, in order	22
8.2	The notified guard	22
8.3	Recording an activity	22
9	Persistence: two plain-text formats	24
9.1	Where the files live	24
9.2	settings.conf: write it, read it, clamp it	24
9.3	When settings are saved	25
10	Statistics: constant memory by construction	26
10.1	The on-disk format	26
10.2	The scanner: fixed buffers, no allocation	26
10.3	The cache: seven days, always	26
10.4	What the totals count	27
10.5	Export and reset	27
11	The AppKit shell	28
11.1	The object graph	28
11.2	refresh(): the frame callback	28
11.3	The main window layout	29
11.4	The action-tag dispatch	29
11.5	Window closing is not quitting	29
12	The egui shell	30
12.1	The two callbacks	30
12.2	effects(): the egui equivalent of refresh()	30
12.3	The Command queue	30
12.4	Closing the window hides it	30
12.5	The dial	31
13	Input: three paths to the same methods	32
13.1	Keyboard	32
13.2	Global hotkeys, without a keyboard tap	32
13.3	The menu bar	33
14	Themes	34
14.1	The five palettes	34
14.2	How a theme reaches the screen	34
14.3	The font	34

15 The release build	35
15.1 The profile	35
15.2 Packaging	35
15.3 What is deliberately not implemented	35
15.4 The size and memory posture	35
16 What the tests actually prove	36
16.1 The test map	36
16.2 The four tests that pin down the subtle rules	37
16.3 What is not tested	37
16.4 Running them	37

1 The one idea that explains everything

Rusty Pomodoro is a Pomodoro timer. That part is easy. The interesting part is that it has *two entirely different user interfaces* — a hand-built AppKit one and an egui one — and the timer logic is shared byte-for-byte between them.

That only works because of a strict layering rule, which is stated in the first line of `src/mac_shell.rs`:

Listing 1: The module's own doc comment states the contract.

```
///! Renderer-free macOS UI. Rust owns timer/storage; AppKit owns controls and drawing.
```

1.1 The three layers

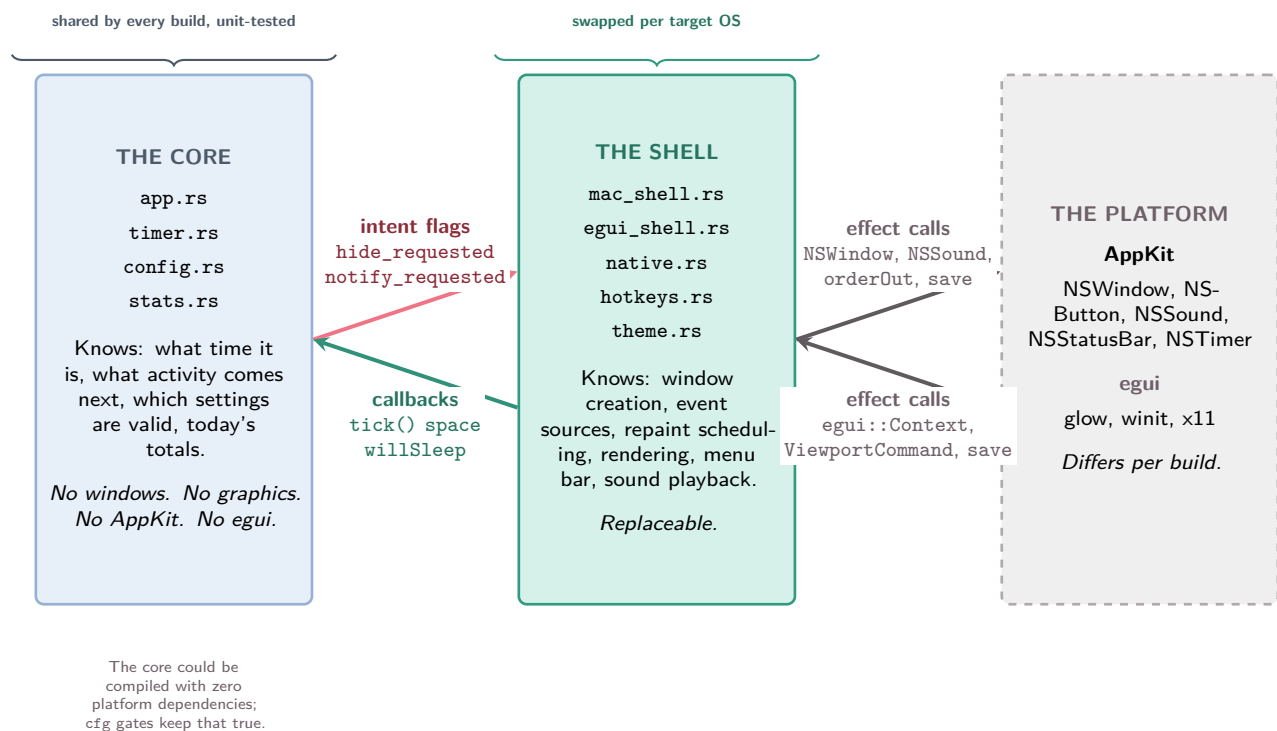


Figure 1: The layering. The core never calls a platform API; it only raises flags. The shell reads them, performs the effect, and clears them with `std::mem::take`.

1.2 The mechanism: five boolean fields

The whole boundary is five public booleans on `App`, from `src/app.rs` lines 22–29:

Listing 2: Every side effect the core wants is a public boolean field.

```
pub hide_requested: bool, // shell should order the window out
pub show_requested: bool, // shell should bring the window forward
pub notify_requested: bool, // shell should play the completion sound
pub tick_requested: bool, // shell should play the per-second tick sound
pub exit_requested: bool, // shell should terminate the process
```

The core never calls `NSBeep()` or `request_repaint`. It only ever writes `true`. The shell then calls `std::mem::take(&mut app.notify_requested)`, which reads the flag *and* resets it to `false` in one step:

Listing 3: Both shells drain the flags identically, at the end of a frame.

```
// src/mac_shell.rs, refresh()
```

```

if std::mem::take(&mut app.hide_requested) { views.window.orderOut(None); }
let show = std::mem::take(&mut app.show_requested);
if std::mem::take(&mut app.notify_requested) { play(app.settings.notification_sound); }
if std::mem::take(&mut app.tick_requested) { play(Sound::Tink); }

```

Because the flag is cleared by the same read that tests it, an intent can never fire twice. And because the core only writes booleans, it never needs to know whether a shell exists at all — the same App compiles inside both shells unmodified.

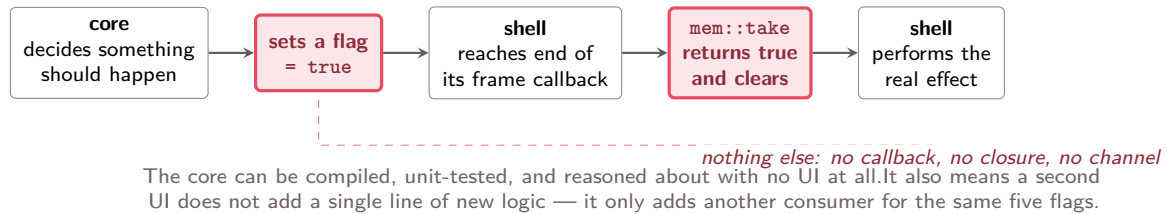


Figure 2: The intent-flag protocol, as a five-step cycle.

1.3 What one second of running looks like

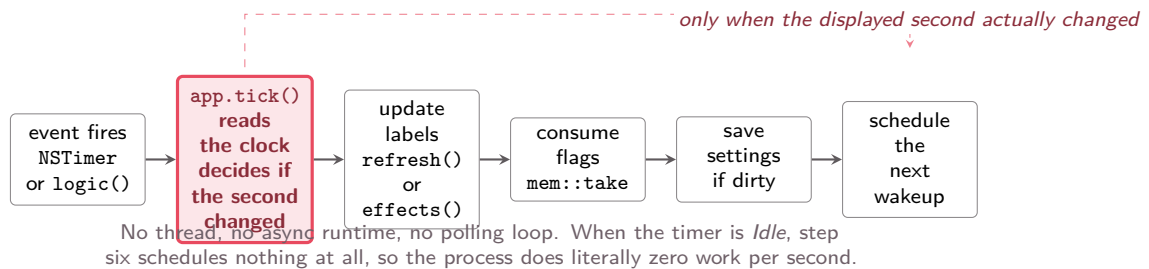
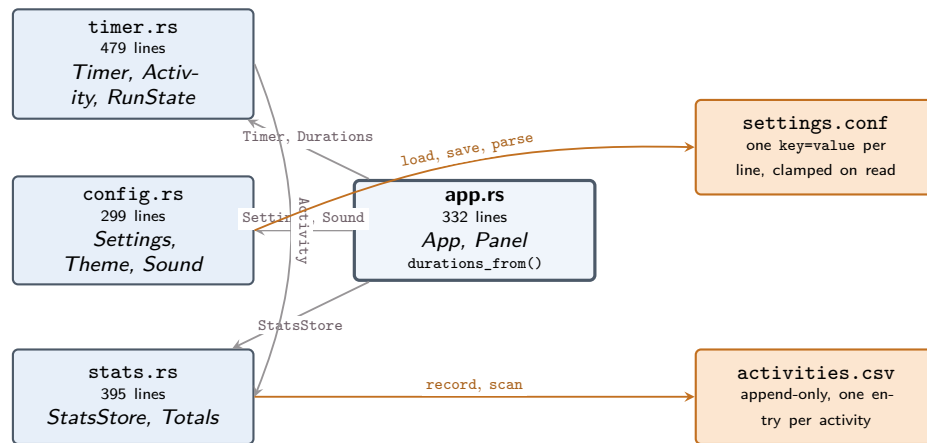


Figure 3: The per-second cycle in the AppKit shell. `refresh()` ends by arming exactly one one-shot NSTimer for the instant the next displayed digit changes.

2 The module map

2.1 The core: four modules and two files on disk

Every arrow below is a real `use crate::...` in the source. Notice the direction of dependency: `timer.rs` and `config.rs` do not know that `App` exists. `App` is the only thing that knows about all of them.



`app.rs` is the only module that knows about all the others.

Figure 4: The platform-free core. `timer.rs` imports only `std::time` and, on macOS, two Mach functions declared by hand. `config.rs` and `stats.rs` touch only `std::fs` and `libc`.

2.2 The shells: two of them, plus their helpers

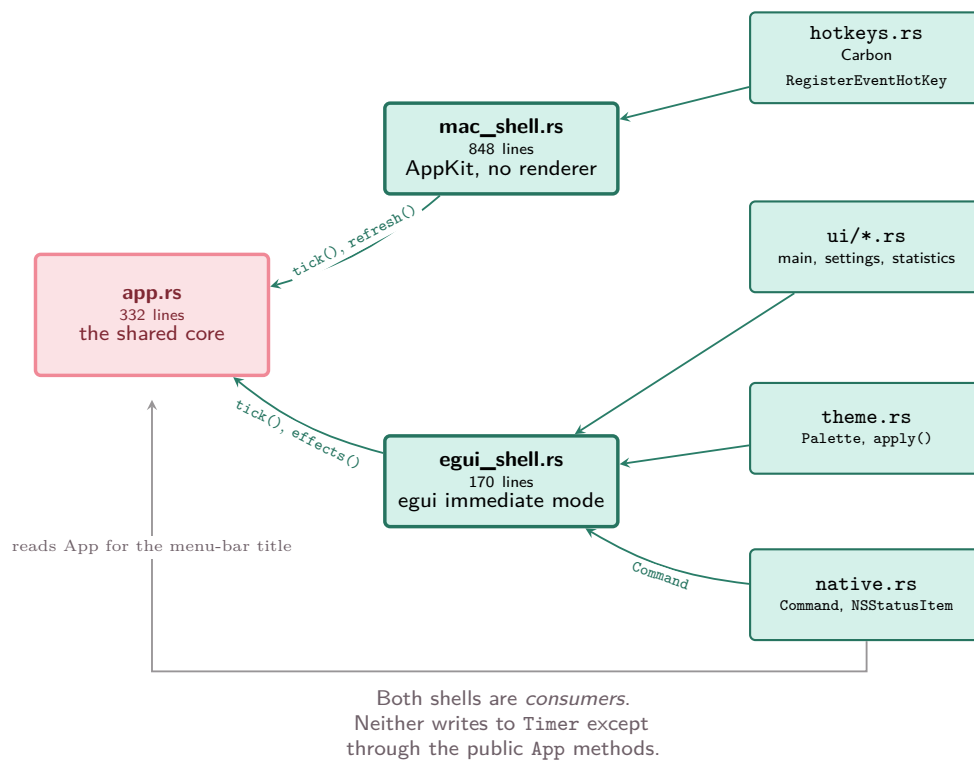


Figure 5: Shell cluster. `native.rs` is a thin AppKit bridge that turns menu clicks and power notifications into an enum value, then queues it for the next frame.

2.3 Line counts

	1000
mac_shell.rs	848
timer.rs	479
stats.rs	395
the entire platform-free core: 1505 lines	
app.rs	332
config.rs	299
native.rs	219
egui_shell.rs	170
ui/*.rs	146
hotkeys.rs	128
theme.rs	118
main.rs	27

Figure 6: Source size by file. The five pastel bars are the core; they compile and test without a window, an image, or a sound card.

2.4 Two shells, side by side

	mac_shell.rs (default on macOS)	egui_shell.rs (-features egui-ui)
Rendering	none — the OS draws the controls	egui immediate mode, glow backend
Event source	one one-shot NSTimer	egui repaint requests
Frame callback	refresh()	effects() , called from logic() and ui()
Menu bar	real NSStatusBar item	native.rs NSStatusItem
Global keys	Carbon RegisterEventHotKey	none (app-focused keys only)
Sound	NSSound::soundNamed	native::play on macOS, silent elsewhere
Window	fixed 380×300, no resize	360×440 minimum, 400×460 default
Themes	one coloured label per theme	five full Palettes via theme.rs
Panels	real NSWindows , built lazily	egui::Windows

Table 1: Both shells drive the identical App. The differences are entirely about how pixels and events arrive, never about what the timer does.

3 One binary, two personalities: the cfg switch

src/main.rs is 27 lines and does nothing but decide which shell to compile. Reading it top to bottom explains the whole build matrix.

Listing 4: src/main.rs, complete.

```
#[cfg(feature = "egui-ui")] fn main() -> eframe::Result { egui_shell::run() }

#[cfg(all(target_os = "macos", not(feature = "egui-ui")))]
fn main() { mac_shell::run(); }

#[cfg(all(not(target_os = "macos"), not(feature = "egui-ui")))]
compile_error!("Build on Windows/Linux with --features egui-ui.");
```

3.1 The matrix this produces

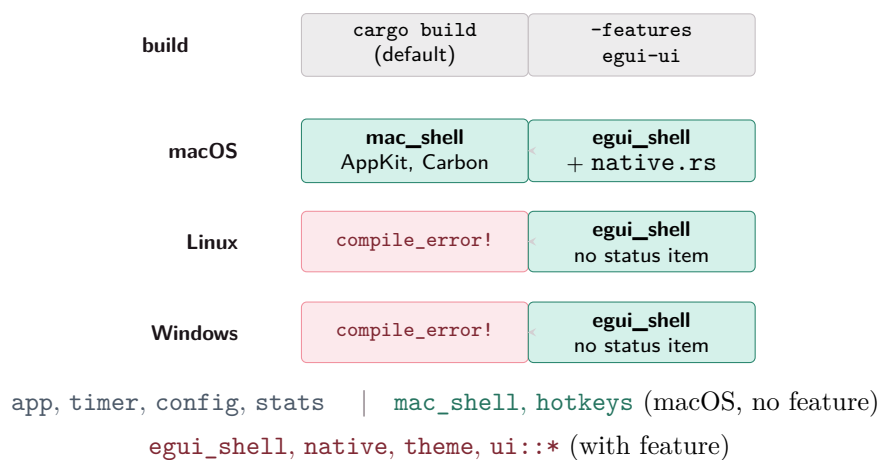


Figure 7: The three build outcomes. The four core modules are compiled in every row; only the shell differs.

3.2 What each module is gated on

Gate on mod	Compiled when	Reason it exists
app, config, stats, timer	always	the portable core
mac_shell, hotkeys	macOS && !egui-ui	the default macOS build
egui_shell, native, theme, ui	feature = "egui-ui"	the cross-platform build

Table 2: The full mod list from main.rs. Nothing is gated on a runtime condition: the choice is made once, at compile time.

3.3 Why a Cargo feature rather than a runtime check

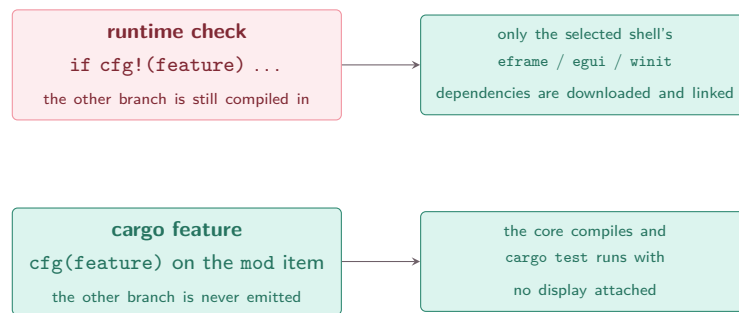


Figure 8: A Cargo feature removes the other branch at compile time, so the default macOS build never links eframe, egui, or winit at all.

Listing 5: Cargo.toml. The whole egui stack is optional, and `default = []` means an ordinary cargo build produces the AppKit app.

```
[features]
default = []
egui-ui = ["dep:eframe", "dep:egui"]

[dependencies]
eframe = { version = "0.36", optional = true, default-features = false, features = ["glow", "x11"] }
egui    = { version = "0.36.2", optional = true, default-features = false }
```

Three deliberate cuts in `default-features = false`. eframe would normally pull in an accessibility stack, a link handler, a web screen reader, and a serde-backed persistence layer; egui would pull in a default font atlas with emoji fallbacks. All of it is disabled. The one 302 KiB Hack-Regular.ttf in assets/ is embedded with `include_bytes!` and installed as the *only* font for both the proportional and monospace families.

4 The timer state machine

`src/timer.rs` is 479 lines and imports exactly one thing: `std::time::Duration`. It is the only file in the project that decides what happens next, and it has no idea a user interface exists.

4.1 Two orthogonal dimensions

The machine has two independent state variables, and almost every bug in a hand-rolled timer comes from confusing them.

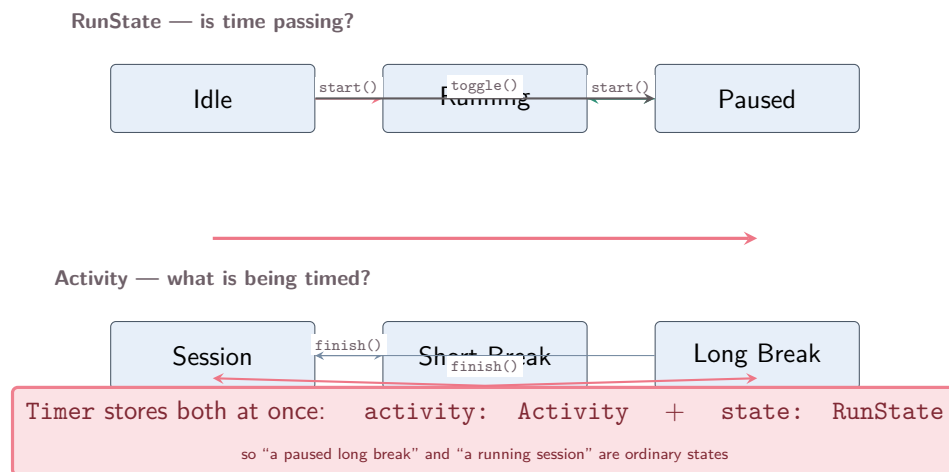


Figure 9: Rusty Pomodoro keeps a *product* state, not a flat enum. “Idle Session” and “Running Session” share the same activity and the same duration but differ entirely in behaviour, so splitting them keeps each rule in one place.

4.2 Time is not stored; it is derived

This is the single most important design decision in `timer.rs`. The struct does *not* keep a countdown that is decremented once a second. It keeps a timestamp and asks the clock:

Listing 6: The four lines that make Timer tick-free.

```

pub fn elapsed(&self) -> Duration {
    match self.started_at {
        Some(start) => self.elapsed + start.elapsed(), // running: frozen + live
        None        => self.elapsed,                  // idle/paused: frozen only
    }
}

pub fn remaining(&self) -> Duration {
    self.duration.saturating_sub(self.elapsed()) // saturates: no underflow
}

```

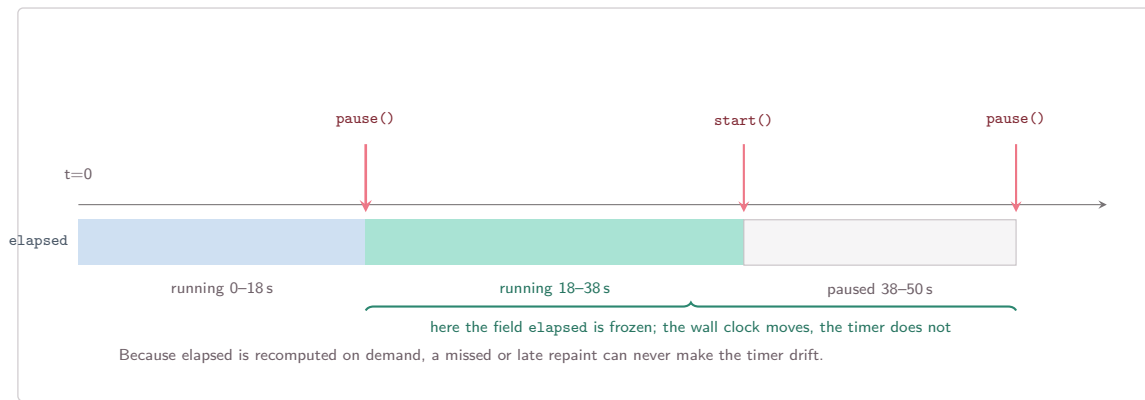


Figure 10: The two-part elapsed time. `elapsed` holds the accumulated total; `started_at` holds the moment the current running stretch began.

Three consequences follow for free:

- **No drift.** There is no “subtract one every tick”, so a late wake-up, a blocked main thread, or a machine that was busy cannot make the timer lose time.
- **Pause is exact.** `pause()` folds the live part into the frozen part and drops the timestamp; the number cannot move afterwards.
- **Overtime falls out.** `remaining()` uses `saturating_sub`, so once `elapsed` passes `duration` it simply pins at zero and `elapsed() - duration()` becomes the overtime count.

4.3 The transition table

Method	From	To	What it does to the fields
<code>start()</code>	Idle / Paused	Running	if Idle <i>and</i> <code>elapsed</code> $\geq$ <code>duration</code> , reset <code>elapsed</code> to 0; set <code>started_at</code> = <code>Clock::now()</code>
<code>pause()</code>	Running	Paused	<code>elapsed</code> += <code>started_at.elapsed()</code> , then <code>started_at</code> = <code>None</code>
<code>toggle()</code>	any	flip	dispatches to <code>pause()</code> or <code>start()</code>
<code>stop()</code>	any	Idle	<code>elapsed</code> = 0, <code>started_at</code> = <code>None</code> . Activity is <i>kept</i>
<code>restart()</code>	any	unchanged	<code>elapsed</code> = 0; re-stamp <code>started_at</code> only if Running
<code>skip()</code>	any	Idle, next activity	<code>advance(Skipped)</code>
<code>finish()</code>	any	Idle, next activity	<code>advance(Finished)</code>
<code>apply_settings()</code>	any	unchanged	folds running time in, then recomputes <code>duration</code>

Table 3: `timer.rs`. `stop` and `restart` are the pair people confuse: `stop` rewinds *and* goes Idle, `restart` rewinds but keeps Running.

4.4 advance: where the Pomodoro rules live

Listing 7: `timer.rs` lines 270–299. This function is the Pomodoro specification; nothing else in the codebase contains a rule.

```
fn advance(&mut self, completion: Completion) -> Completion {
    let finished = self.activity;
```

```

if completion == Completion::Finished && finished == Activity::Session {
    self.completed_sessions = self.completed_sessions.saturating_add(1);
}
let next = match finished {
    Activity::Session => {
        let cycle = self.settings.long_break_cycle.max(1);
        let due_long = completion == Completion::Finished
            && self.completed_sessions > 0
            && self.completed_sessions.is_multiple_of(cycle);
        if self.settings.disable_breaks { Activity::Session }
        else if due_long && !self.settings.disable_long_breaks { Activity::LongBreak }
        else { Activity::ShortBreak }
    }
    Activity::ShortBreak | Activity::LongBreak => Activity::Session,
};
self.activity = next;
self.duration = duration_for(next, self.settings);
self.elapsed = Duration::ZERO;
self.started_at = None;
self.state = RunState::Idle;
completion
}

```

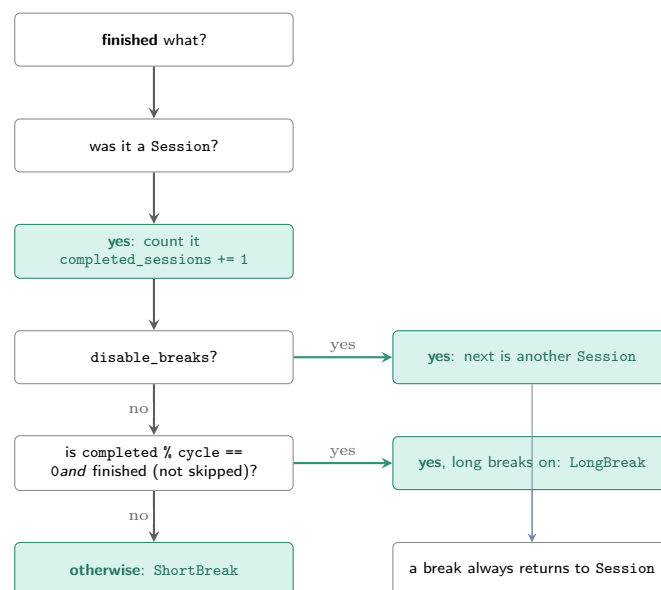


Figure 11: The decision tree of `advance()`. The highlighted boxes are the only places in the whole codebase where the answer to “what comes next?” is decided.

The subtle line. `due_long` requires `completion == Completion::Finished`. If the user skips a session that happens to land on a cycle boundary, the long break is *not* scheduled — and `completed_sessions` is not incremented either, so the cycle counter stays aligned. The test `skipped_session_at_cycle_boundary_does_not_schedule_long_break` in `timer.rs` exists precisely to pin this down.

5 The Pomodoro cycle, end to end

5.1 Normal case: a 4-session cycle

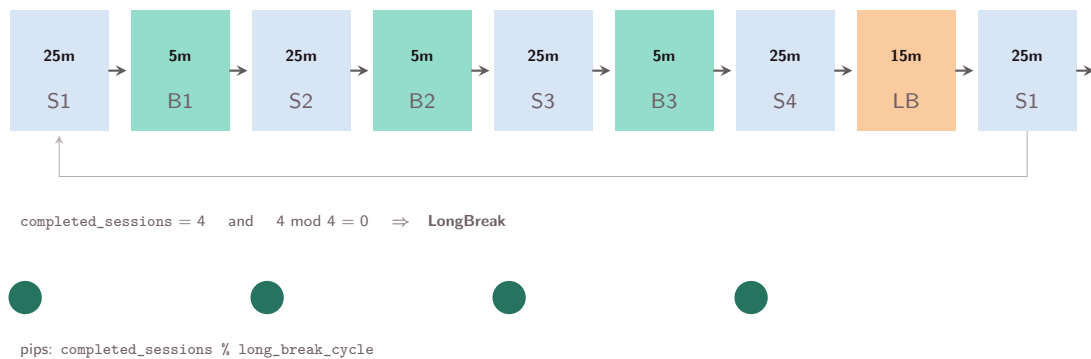


Figure 12: With the default settings (`long_break_cycle = 4`) the fourth finished session earns a long break. Skipping any session breaks the chain — the counter is only incremented by `Completion::Finished`.

5.2 The three switches that reshape the cycle

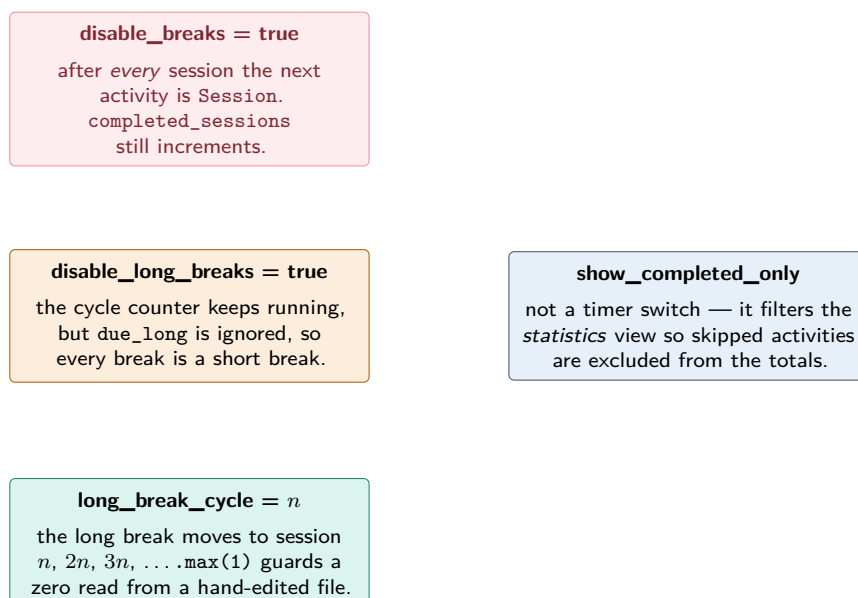


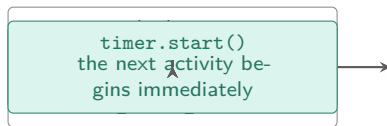
Figure 13: Four settings that change the shape of the cycle. The first three are read inside `advance()` on every transition; the fourth only affects the numbers shown in the statistics panel.

5.3 Auto-start, and who decides it

Listing 8: `app.rs`, `complete_current()`. Note that the core decides *whether* to start the next activity, but never starts it itself unless asked.

```
pub fn complete_current(&mut self) {
    let finished = self.timer.activity();
    self.status = format!("{}", finished, finished.label());
    self.record(true); // write the CSV row first
    self.timer.finish(); // advance; state becomes Idle
    self.notified = false;
    let auto = if self.timer.activity().is_break() { // note: the NEXT activity
        self.settings.auto_start_break
    } else {
        self.settings.auto_start_session
    }
}
```

```
};
if auto { self.timer.start(); }
}
```



With overtime on, the chain stops here.
timer keeps running, the display shows +MM:SS,
and the user ends it with Finish or the F key.

Figure 14: End-of-activity path with overtime off. With overtime on, the completion is deferred and the activity stays running past zero.

5.4 Overtime: the display flips sign

Listing 9: `app.rs`. Overtime is a *display* mode plus a deferred completion; the timer itself never changes state.

```
pub fn display_seconds(&self) -> u64 {
    if self.is_overtime() {
        self.timer.elapsed().saturating_sub(self.timer.duration()).as_secs()
    } else {
        self.timer.remaining_display().as_secs()
    }
}
pub fn is_overtime(&self) -> bool {
    self.settings.overtime
    && self.timer.remaining().is_zero()
    && self.timer.state() != RunState::Idle
}
```

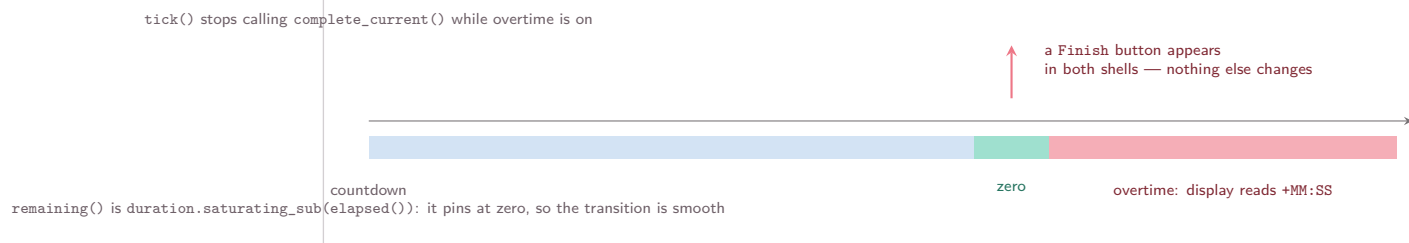


Figure 15: Overtime is a display convention, not a fourth RunState.

6 The clock, and the display it feeds

6.1 Two clocks behind one type name

`timer.rs` defines a private `Clock` enum with one variant per platform. This is the only piece of the core that is platform-conditional, and it is 40 lines.

Listing 10: `timer.rs` lines 4–59, abridged.

```
#[derive(Clone, Copy, Debug)]
struct Clock {
    #[cfg(target_os = "macos")]    ticks: u64,           // mach_continuous_time
    #[cfg(not(target_os = "macos"))] instant: std::time::Instant,
}

impl Clock {
    fn now() -> Self { /* one branch per platform */ }
    fn elapsed(self) -> Duration {
        #[cfg(target_os = "macos")]
        {
            static SCALE: OnceLock<u32,u32> = OnceLock::new();
            let &(numer, denom) = SCALE.get_or_init(|| { /* mach_timebase_info */ });
            let nanos = (Clock::now().ticks - self.ticks) as u128 * numer as u128
                / denom as u128;
            Duration::from_nanos(nanos.min(u64::MAX as u128) as u64)
        }
        #[cfg(not(target_os = "macos"))]
        { self.instant.elapsed() }
    }
}

unsafe extern "C" { fn mach_continuous_time() -> u64; fn mach_timebase_info(..) -> i32; }
```

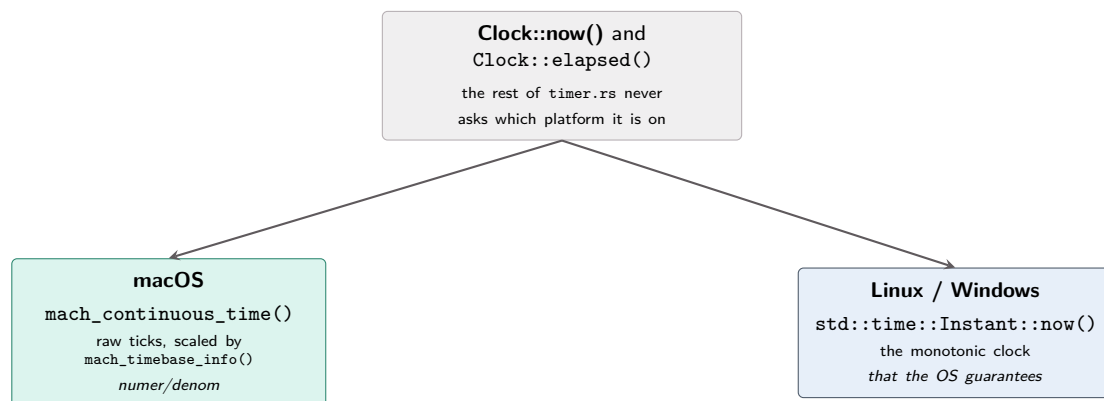


Figure 16: `Clock` is the entire platform abstraction of the core.

Why `mach_continuous_time` and not `mach_absolute_time`? The “continuous” variant keeps counting while the machine is asleep, which is what a Pomodoro timer wants: if the lid is shut for ten minutes, the session really did advance by ten minutes and `App::sleep()` decides separately whether to pause it. The timebase conversion is cached in a `OnceLock` because it is a constant for the lifetime of the process.

6.2 Rounding: why a fresh timer shows 25:00, not 24:59

Two tiny functions carry all of the display logic, and both exist because the alternative was a visible one-second jitter.

Listing 11: Round up, so the label is stable for a whole second.

6.3 How each shell spends that delay

	mac_shell.rs	egui_shell.rs
Idle timer	no timer armed	no repaint scheduled
Running	refresh() ends with a one-shot NSTimer	schedule_repaint() calls ctx.request_repaint_after(d)
Tolerance	setTolerance(0.02) — the OS may fire 20 ms late	exact; egui coalesces into the next frame
Clamped to	max(0.01 s)	n/a
Repeat?	repeats: false, then re-armed in refresh()	re-armed every frame

Table 4: Both shells converge on the same rule: schedule exactly one wake-up, at the next visible change. Neither one ever busy-waits.

Listing 13: app.rs lines 172–176: the whole scheduling policy is three lines.

```
[cfg(feature = "egui-ui")]
pub fn schedule_repaint(&self, ctx: &egui::Context) {
    if self.timer.state() == RunState::Running {
        ctx.request_repaint_after(self.timer.next_display_change());
    }
}
```

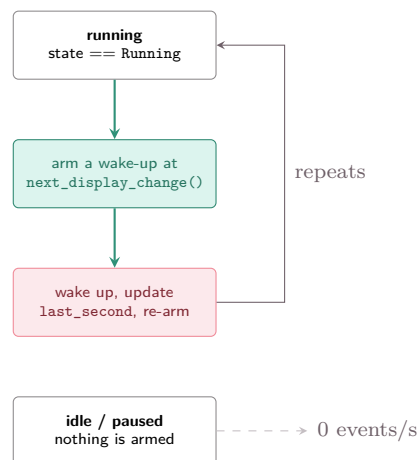


Figure 18: The repaint loop, which is really just a one-node cycle.

7 The intent-flag pattern, in full

This is the pattern that makes the whole project work, so it is worth seeing every place it is used. There are exactly five flags, and each one has a producer in the core and a consumer in the shell.

7.1 The five flags, their producers, their consumers

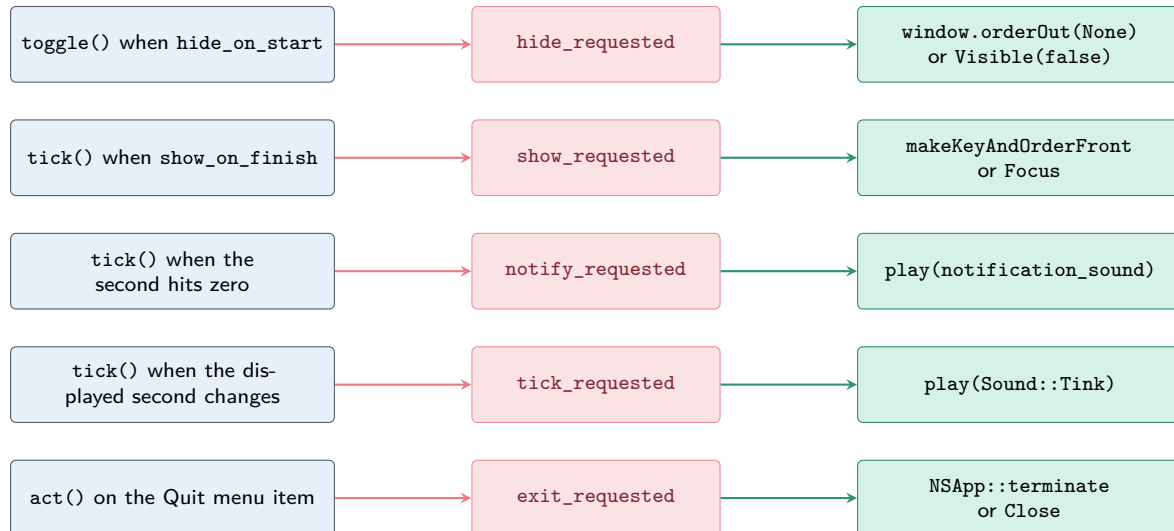


Figure 19: Every flag is a one-way trip: the core sets it, the shell reads-and-clears it. There is no callback, no closure, no channel, and no shared mutable state beyond the flag itself.

7.2 Why `mem::take` and not `if`

the flag can never
be observed in the
“set but not yet handled” state

Figure 20: `std::mem::take` replaces the value with `Default::default()` and returns the old one, so the test and the reset are a single operation.

Listing 14: The two versions side by side. The second is what both shells use.

```
// naive: two steps, and a window in between
if app.notify_requested {
    play(sound);
    app.notify_requested = false;
}

// mem::take: one step, no window
if std::mem::take(&mut app.notify_requested) {
    play(sound);
}
```

7.3 The one flag that is not a flag

`exit_requested` is set by the shell itself (the Quit menu item calls `NSApplication::terminate` directly in the AppKit shell), so it is the only one whose producer is not the core. Everything else follows the rule.

7.4 Sleep and wake

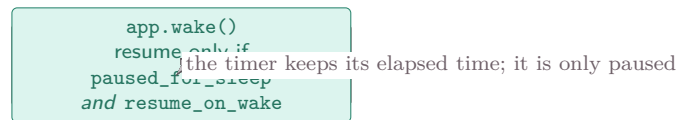


Figure 21: Sleep handling is a two-step handshake. The `paused_for_sleep` field is what stops a wake from resuming a timer the user had paused themselves.

Listing 15: `app.rs` lines 177–188. The asymmetry is the point.

```
pub fn sleep(&mut self) {
    if self.settings.pause_on_sleep && self.timer.state() == RunState::Running {
        self.timer.pause();
        self.paused_for_sleep = true;
    }
}

pub fn wake(&mut self) {
    if self.paused_for_sleep && self.settings.resume_on_wake { self.timer.start(); }
    self.paused_for_sleep = false;
}
```

8 tick(): the one function that runs every second

Everything the app does once a second goes through `App::tick()`. It is 23 lines and it is the only place where the core decides that something has changed.

Listing 16: `app.rs` lines 133–155, complete.

```
pub fn tick(&mut self) {
    if self.timer.state() != RunState::Running { return; }           // (1) idle: do nothing
    if self.timer.remaining().is_zero() {                             // (2) the second hit zero
        if !self.notified {
            self.notified = true;
            self.notify_requested = true;
            if self.settings.show_on_finish { self.show_requested = true; }
        }
        if !self.settings.overtime { self.complete_current(); }     // (3) or keep running
    }
    let second = self.display_seconds();                               // (4) did the label
    change?
    if second != self.last_second {
        self.last_second = second;
        self.tick_requested = self.settings.ticking_sound
            && !(self.settings.mute_ticking_on_break && self.timer.activity().is_break());
    }
}
```

8.1 The four decisions, in order

yes → raise tick_requested
if tick_requested is raised
for this activity

Figure 22: `tick()` as a decision tree. Note that step (4) runs even when the activity just completed, so the tick sound and the completion sound cannot collide.

8.2 The notified guard

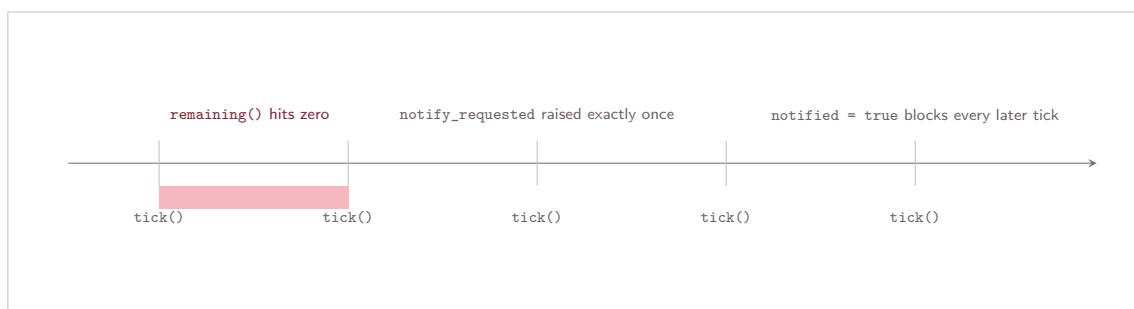


Figure 23: Without the `notified` field, every subsequent tick would re-raise `notify_requested` and the shell would play the sound once per second forever.

8.3 Recording an activity

*self.cache.borrow_mut()
= None
the 7-day cache is invalidated

Figure 24: Recording is a single append plus a cache invalidation. There is no read-modify-write of the file, so a crash cannot corrupt the log.

Listing 17: `app.rs` lines 100–110. The `max` is what makes a completed session count as a full session even if the user let it run into overtime.

```
fn record(&mut self, completed: bool) {
    let elapsed = self.timer.elapsed().as_secs();
    let seconds = if completed {
        elapsed.max(self.timer.duration().as_secs())
    } else {
        elapsed
    };
    if let Err(error) = self.stats.record(self.timer.activity(), seconds, completed) {
        self.status = format!("Activity save failed: {error}");
    }
}
```

9 Persistence: two plain-text formats

The app has no database, no serialization framework, and no `serde`. Both stores are plain text with a hand-written parser, and both are written atomically.

9.1 Where the files live

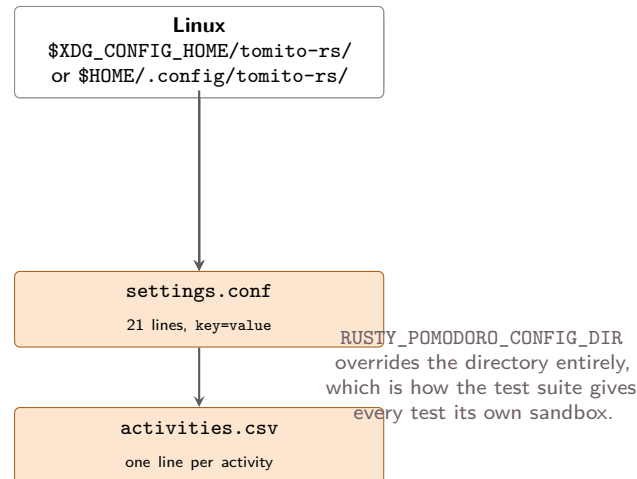


Figure 25: `config::config_dir()` picks the platform directory. The override environment variable is the only configuration the app reads at startup.

9.2 settings.conf: write it, read it, clamp it

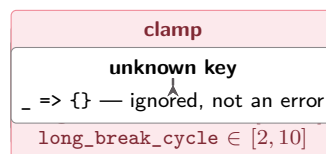


Figure 26: The settings pipeline. Every numeric field is range-checked on read, so a hand-edited file cannot produce a 9999-minute session.

Field	Default	Notes
<code>session_minutes</code>	25	clamped to 1–180
<code>short_break_minutes</code>	5	clamped to 1–60
<code>long_break_minutes</code>	15	clamped to 1–120
<code>long_break_cycle</code>	4	clamped to 2–10
<code>auto_start_session</code>	true	read by <code>complete_current()</code>
<code>auto_start_break</code>	true	read by <code>complete_current()</code>
<code>disable_breaks</code>	false	read by <code>advance()</code>
<code>disable_long_breaks</code>	false	read by <code>advance()</code>
<code>show_completed_only</code>	true	filters the statistics view
<code>hide_on_start</code>	false	raises <code>hide_requested</code>
<code>keep_in_front</code>	false	window level
<code>hide_on_launch</code>	false	applied once in <code>App::new</code>
<code>show_on_finish</code>	true	raises <code>show_requested</code>
<code>overtime</code>	false	display mode + deferred completion
<code>pause_on_sleep</code>	true	<code>app.sleep()</code>
<code>resume_on_wake</code>	true	<code>app.wake()</code>
<code>ticking_sound</code>	false	per-second tick
<code>mute_ticking_on_break</code>	true	suppresses the tick during breaks
<code>theme</code>	Flamingo	one of five
<code>notification_sound</code>	Ping	one of twelve

Table 5: All 21 settings fields. Every one of them is read from exactly one place in the code, which is why the struct can be Copy.

9.3 When settings are saved

```
settings_dirty = false
```

Figure 27: Saving is deferred, not immediate. The egui shell coalesces writes so that dragging a slider does not hit the disk on every pixel.

Why `fs::rename`? On all three target platforms `rename` within the same directory is atomic: a reader sees either the old file or the new one, never a partial write. The `.tmp` suffix also means a crashed write leaves a stray file instead of a truncated `settings.conf`.

10 Statistics: constant memory by construction

`stats.rs` has a one-line summary in its module doc: “*Streaming CSV storage and a fixed seven-day cache: memory does not grow with history.*” Everything in the file exists to make that sentence true.

10.1 The on-disk format

```
epoch_seconds, duration_seconds, activity, completed
```

```
1759218604, 1500, 0, 1
```

```
17592187544, 300, 1, 1
```

```
17592187844, 120, 0, 0
```

```
17592187964, 60, 2, 0
```

```
activity: 0 = Session, 1 = ShortBreak, 2 = LongBreak
```

```
completed: 1 if the activity ran to the end, 0 if it was skipped or stopped
```

Figure 28: One line per activity. The file is append-only, so the only write is `writeln!` at the end.

10.2 The scanner: fixed buffers, no allocation

```
valid?
visit(entry)
otherwise silently skipped
```

Figure 29: `scan()` uses two fixed stack buffers. A corrupt multi-megabyte line costs one boolean, not a allocation.

Listing 18: `stats.rs` lines 190–192. The two buffers are the whole memory story.

```
let mut chunk = [0; 8192]; // read buffer, on the stack
let mut line = [0; 256]; // line buffer, on the stack
let (mut len, mut overflow) = (0, false);
```

10.3 The cache: seven days, always

```
invalidate
*self.cache.borrow_mut()
    .as_mut()
    = None
on record() and reset()
```

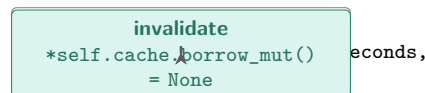
Figure 30: The cache is a `RefCell<Option<Cache>`, so `StatsStore` can stay `&self` in `record()` and still invalidate itself.

10.4 What the totals count

Field	Incremented when	Notes
sessions	a <code>Session</code> entry with <code>completed = true</code>	skipped sessions do not count
breaks	a break entry with <code>completed = true</code>	short and long combined
long_breaks	a <code>LongBreak</code> entry with <code>completed = true</code>	a subset of <code>breaks</code>
skipped	any entry with <code>completed = false</code>	regardless of activity
session_seconds	every <code>Session</code> entry	completed or not
break_seconds	every break entry	completed or not

Table 6: `Totals::add`. The asymmetry is deliberate: seconds are always counted, but the counters only count completed activities.

10.5 Export and reset



```

invalidate
*self.cache.borrow_mut() = None, seconds,

```

Figure 31: Both operations are atomic and both invalidate the cache. The export guard is the only place in the file that returns an error for a reason other than I/O.

11 The AppKit shell

`mac_shell.rs` is 848 lines and contains no rendering code at all. It builds real AppKit objects with absolute frames, and lets the operating system draw them.

11.1 The object graph

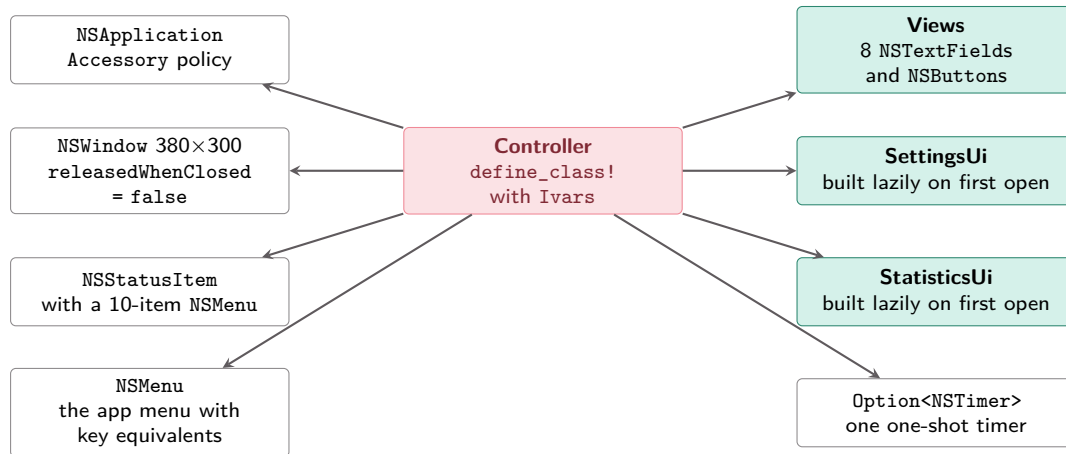


Figure 32: Every AppKit object is a `Retained<T>` held in a struct field. Nothing is leaked, and nothing is freed early: `objc2` inserts the release calls.

11.2 `refresh()`: the frame callback

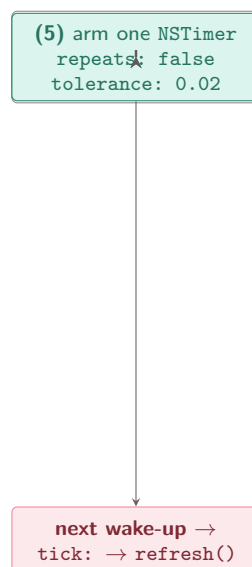


Figure 33: `refresh()` is the only place that touches the views. It is called from the timer, from every button action, and from the sleep/wake notifications.

11.3 The main window layout

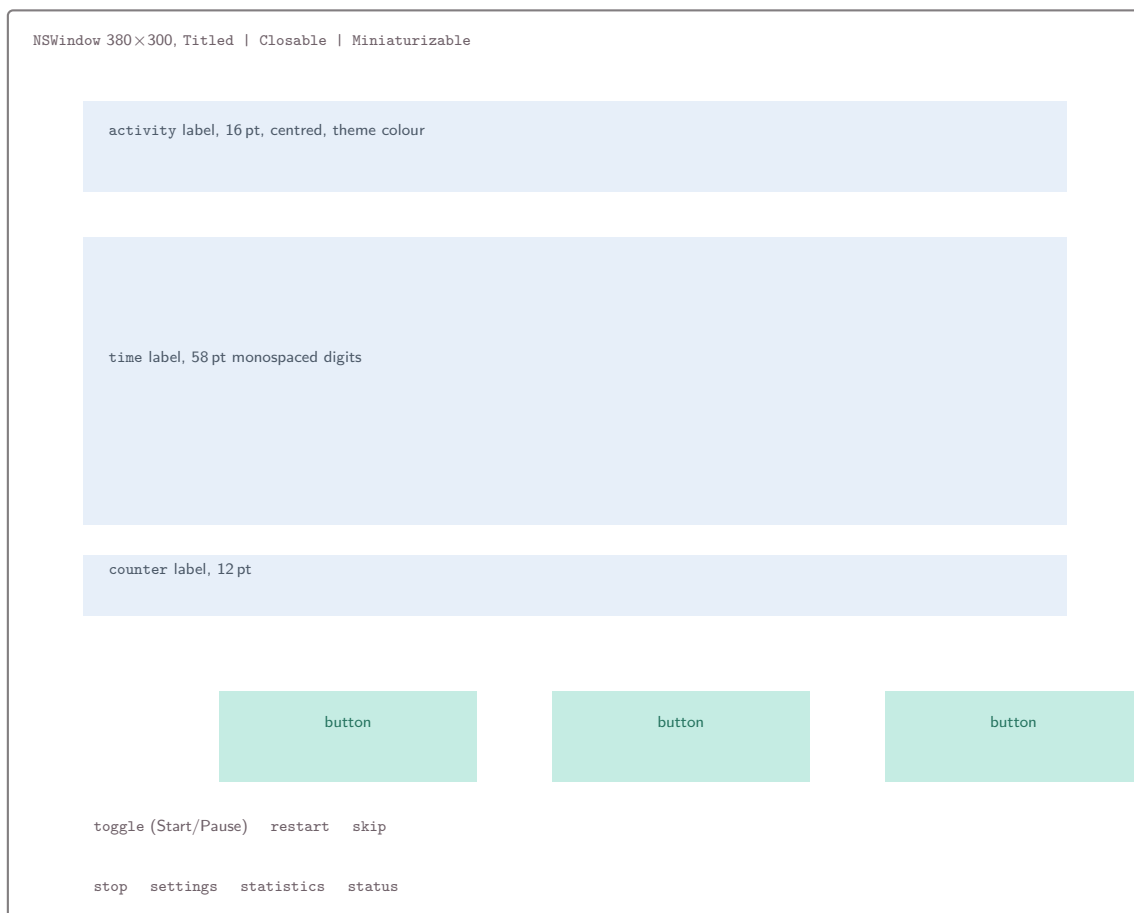


Figure 34: The main window is built with absolute NSRect frames in `build()`. There is no auto-layout, so the positions are exact and the window cannot be resized.

11.4 The action-tag dispatch

Every button and menu item calls the same `act:` selector, and the `tag` field decides what happens. This is the entire input layer.

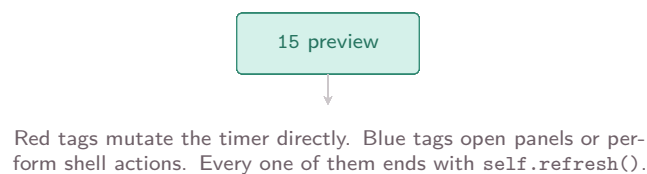


Figure 35: Sixteen tags, one selector. Adding a control means adding a tag and a line in the `match`.

11.5 Window closing is not quitting

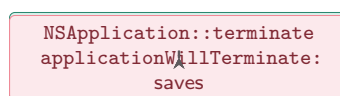


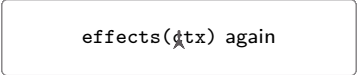
Figure 36: `applicationShouldTerminateAfterLastWindowClosed` returns `false`, so closing the window never quits the app. The menu bar item is the way back.

12 The egui shell

`egui_shell.rs` is 170 lines and contains no timer logic. It is a translation layer: it turns egui events into `App` method calls, and it turns `App`'s intent flags into egui commands.

12.1 The two callbacks

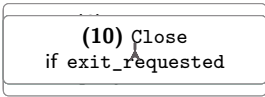
egui's `App` trait has two methods, and the split between them is the whole design.



```
effects(&tx) again
```

Figure 37: `logic()` mutates, `ui()` draws. `effects()` runs in both, so a flag raised while drawing is still handled in the same frame.

12.2 `effects()`: the egui equivalent of `refresh()`



```
(10) Close  
if exit_requested
```

Figure 38: Ten steps, in order. Steps 1–2 are idempotent state syncs, 3–6 are the intent flags, and 7–10 are the shell's own bookkeeping.

12.3 The Command queue

`native.rs` turns AppKit events into an enum and queues them. The shell drains the queue once per frame.



```
Wake
```

The menu bar and the power notifications are the only two producers. Both call `context.request_repaint()` after pushing, so the queue is never stale.

Figure 39: Twelve `Command` variants. The tag on each `NSMenuItem` is the discriminant, so adding a menu item is a one-line change in `native.rs` and one `match` arm in `egui_shell.rs`.

12.4 Closing the window hides it



```
exit_requested = true  
effects() sends Close
```

Figure 40: The egui shell reproduces the AppKit shell's behaviour: closing hides, quitting terminates. The `CancelClose` command is what makes the first part work.

12.5 The dial

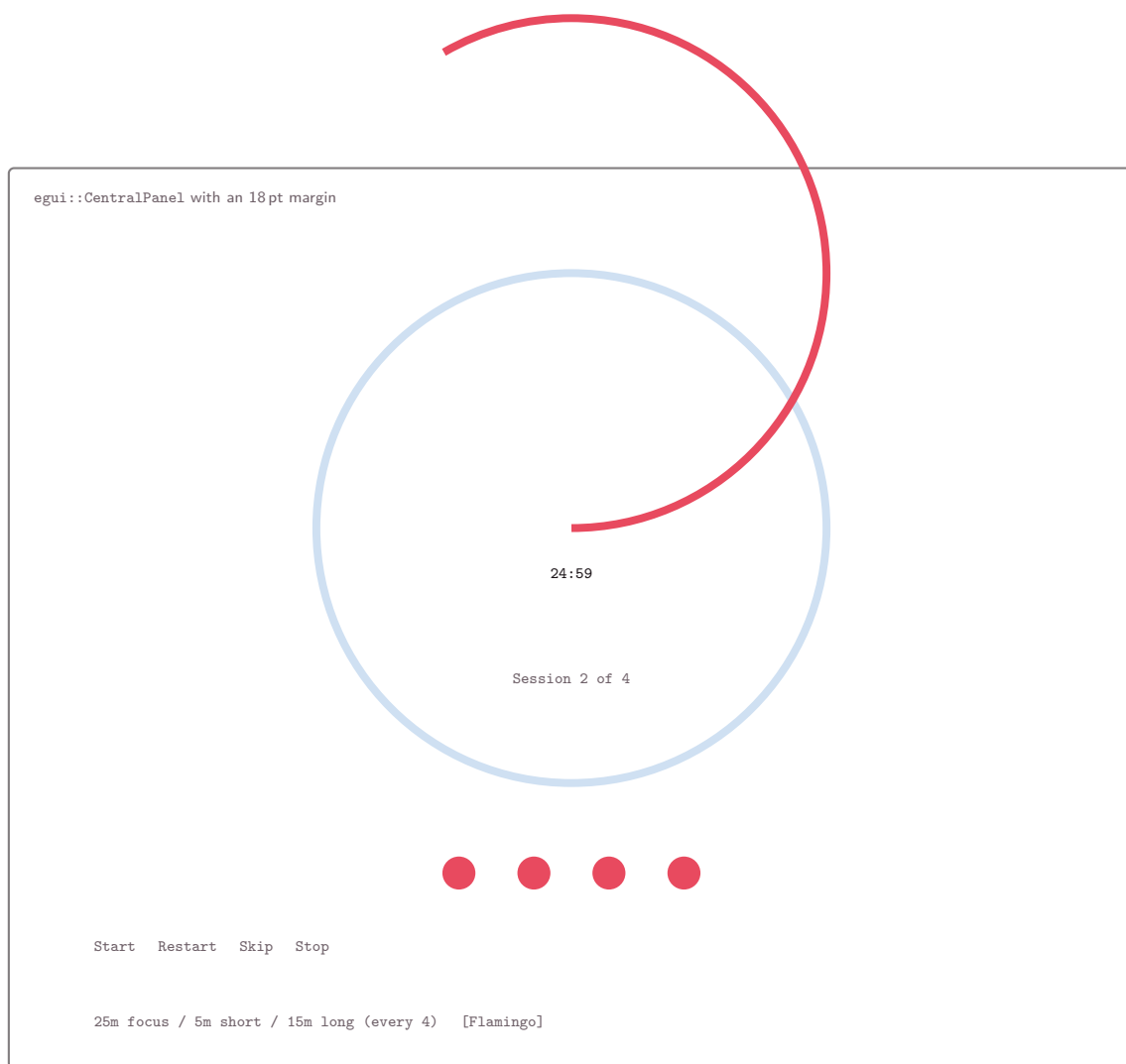


Figure 41: The main panel is drawn with `painter` calls, not widgets. The progress arc is a polyline of 96 points, which avoids both polygon triangulation and an image texture.

13 Input: three paths to the same methods

There are exactly three ways to reach the timer: keyboard, menu bar, and buttons. All three end at the same **App** methods.

13.1 Keyboard

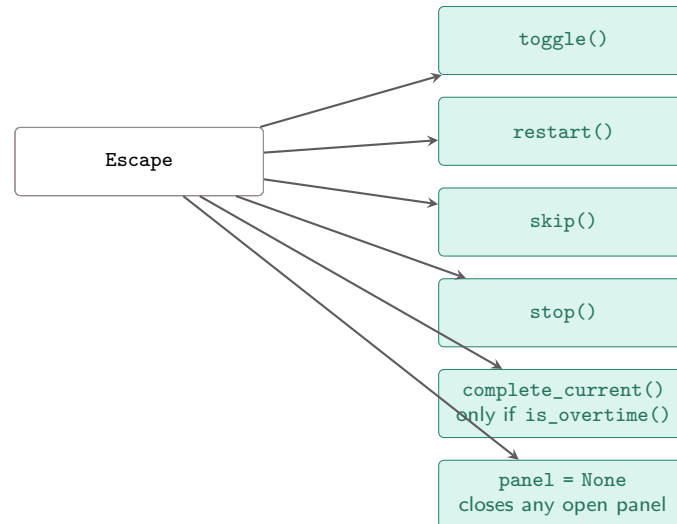


Figure 42: Six keys. Escape is handled first and closes the panel; the other five are ignored while a panel is open or while egui wants the keyboard.

Listing 19: `app.rs` lines 190–216. The guard at the top is what keeps the timer from reacting while the user is typing in a settings field.

```

pub fn handle_keys(&mut self, ctx: &egui::Context) {
    let escape = ctx.input(|i| i.key_pressed(egui::Key::Escape));
    if escape { self.panel = Panel::None; self.confirm_reset = false; }
    if ctx.egui_wants_keyboard_input() || self.panel != Panel::None { return; }
    ctx.input(|i| {
        if i.key_pressed(egui::Key::Space) { self.toggle(); }
        if i.key_pressed(egui::Key::R)     { self.restart(); }
        if i.key_pressed(egui::Key::S)     { self.skip(); }
        if i.key_pressed(egui::Key::X)     { self.stop(); }
        if i.key_pressed(egui::Key::F) && self.is_overtime() { self.complete_current(); }
    });
}

```

13.2 Global hotkeys, without a keyboard tap

The AppKit shell registers five Carbon hotkeys. They work even when the app is not focused, and they need no permission prompt because they are registered on the application's own event target.

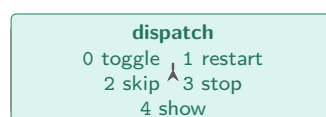
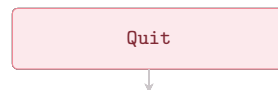


Figure 43: `hotkeys.rs` is 128 lines of raw Carbon FFI. The TMRs signature is a magic value that stops the handler from reacting to some other app's hotkey.

Key code	Key	Modifiers	Action
49	Space	Control + Option	<code>toggle()</code>
15	R	Control + Option	<code>restart()</code>
1	S	Control + Option	<code>skip()</code>
7	X	Control + Option	<code>stop()</code>
17	T	Control + Option	<code>show()</code>

Table 7: The five registered hotkeys. The T key is the “show timer” command; it is the only one that does not mutate the timer.

13.3 The menu bar



The same ten items exist in both shells. In the egui shell they are produced by `native.rs`; in the AppKit shell by `mac_shell.rs`. The tags are identical, so the match in `act:` and the match in `logic()` agree by construction.

Figure 44: Ten menu items. The first item’s title tracks the run state, so the menu bar always says Start, Pause, or Resume.

14 Themes

Five themes, each a **Palette** of five colours. The AppKit shell uses one of them for a single label; the egui shell pushes all five into the global style.

14.1 The five palettes

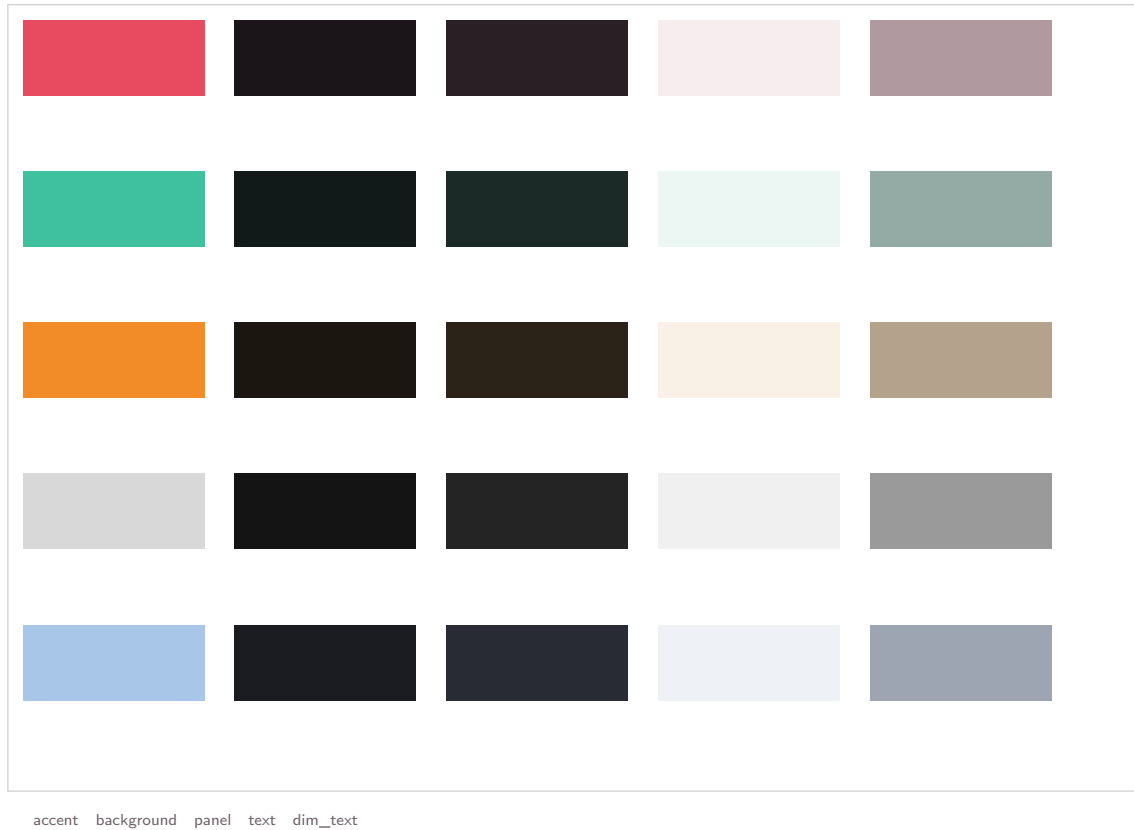


Figure 45: The five palettes from `theme.rs`. Every one of them is dark, and every accent is bright enough to read against its own background.

14.2 How a theme reaches the screen



Figure 46: One `Palette` struct, two very different applications of it. The egui path rewrites the global style; the AppKit path sets one colour on one label.

14.3 The font



Figure 47: The font is the only binary asset in the project. There are no icons, no images, and no sound files.

15 The release build

15.1 The profile

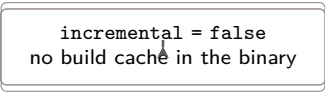


Figure 48: Six settings, all aimed at the same goal. The README is explicit that these should be measured, not trusted: `ls -lh target/release/rusty-pomodoro` and an idle memory reading are the real evidence.

15.2 Packaging

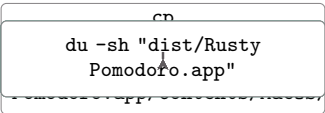


Figure 49: `scripts/package-macos.sh` is five commands. There is no DMG, no notarisation, and no installer.

15.3 What is deliberately not implemented

Feature	Why it is absent
Menu-bar status item (egui build)	<code>native.rs</code> provides one, but the egui build does not use it
AppleScript command suite	needs a scripting dictionary and <code>sdef</code> tooling
Global hotkeys (egui build)	Carbon only; the egui build has app-focused keys only
Sleep and wake handling (egui build)	<code>native.rs</code> provides it, but only on macOS
Audio playback	sound names are stored and <code>NSSound::soundNamed</code> is called, but no audio files are bundled
Wayland	the <code>eframe</code> features are <code>glow</code> and <code>x11</code> only

Table 8: The README’s “Not implemented” list. Each item is a platform API that lies outside the egui scope of this build, not a missing piece of the core.

15.4 The size and memory posture

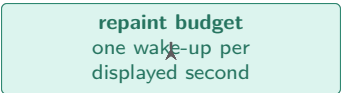


Figure 50: Seven independent decisions that each shave something off the binary. None of them is a compromise in behaviour: the timer still works identically.

16 What the tests actually prove

There are 31 tests. They are all in `#[cfg(test)]` modules inside the files they test, which means they compile with the same `cfg` gates as the code.

16.1 The test map

`timer.rs`

14 tests

- `starts_idle_on_a_session` — initial state is `Idle/Session` with the full duration
- `session_cycles_into_short_then_long_breaks` — the 4th finished session earns a `LongBreak`
- `skip_moves_on_without_counting_a_session` — skipping does not increment the counter
- `disable_breaks_jumps_straight_back_to_a_session` — the setting is honoured in `advance()`
- `disable_long_breaks_keeps_only_short_breaks` — the cycle counter still runs
- `pause_stops_elapsed_time` — elapsed is frozen across a pause
- `stop_resets_the_current_activity` — elapsed goes to zero, activity is kept
- `restart_clears_progress_but_keeps_running` — the run state survives a restart
- `progress_and_remaining_track_the_activity` — both are derived from elapsed
- `remaining_display_rounds_up_whole_seconds` — a fresh timer shows 25:00, not 24:59
- `exact_and_fractional_rounding_and_deadlines` — `next_display_change()` returns the complement
- `skipped_session_at_cycle_boundary_does_not_schedule_long_break` — the `completion == Finished` check
- `running_duration_change_clamps_actual_progress` — `saturating_sub` prevents underflow
- `new_settings_reshape_the_current_activity` — `apply_settings()` recomputes the duration

`app.rs`

5 tests

- `completion_records_and_auto_starts_break` — the CSV row is written before the advance
- `skip_does_not_count_completion_or_long_break` — skipped sessions stay out of the long-break cycle
- `sleep_only_resumes_previously_running_timer` — the `paused_for_sleep` handshake
- `settings_dirty_and_hide_request` — the dirty flag and the hide flag are independent
- `expiration_and_overtime_notify_once` — the notified guard

`config.rs`

3 tests

- `round_trip` — `parse(serialize(s)) == s`
- `invalid_input_and_ranges` — out-of-range values are clamped, garbage is ignored
- `old_theme_keys_still_load` — `theme=mint` and `theme=mono` both parse

`stats.rs`

5 tests

- `totals_filter_incomplete_and_cover_seven_days` — the 7-day window and the completed-only filter
- `export_has_five_columns_and_original_timestamp` — the export format and the self-overwrite guard
- `oversized_and_malformed_lines_are_ignored` — the 256-byte line buffer
- `record_and_reset_invalidate_cache` — the cache cannot serve stale totals
- `calendar_and_duration_format` — the day-index maths and the 1h 5m format

`ui/main.rs` and `theme.rs`

2 tests

- `accent_comes_from_the_palette` — the palette is wired into the UI
- `every_theme_kind_maps_to_a_palette` — all five themes produce a valid palette

Every test creates its own temporary directory and removes it at the end, so the suite can run in parallel and never touches the real `$HOME`. The two green entries are the only tests that touch egui types; the rest run with no UI compiled in.

16.2 The four tests that pin down the subtle rules

Test	What it would catch
<code>skipped_session_at_cycle_boundary_does_not_schedule_long_break</code>	moving the <code>completed_sessions</code> increment into the shared path, or dropping the <code>completion == Finished</code> check from <code>due_long</code>
<code>expiration_and_overtime_notify_once</code>	removing the <code>notified</code> guard, or re-raising <code>show_requested</code> on every tick
<code>sleep_only_resumes_previously_running_timer</code>	resuming on wake unconditionally, which would restart a timer the user had paused themselves
<code>running_duration_change_clamps_actual_progress</code>	a settings change making <code>remaining()</code> underflow instead of saturating

Table 9: Four tests, each guarding a single line that would otherwise be easy to break.

16.3 What is not tested



Figure 51: The gap is structural: the shells are thin translation layers over platform APIs, and the logic they translate is fully covered.

16.4 Running them

Listing 20: The whole suite, and the two builds it must pass on.

```
cargo test                # core + AppKit shell
cargo test --features egui-ui # core + egui shell
```

One suite, two binaries. Because the core is shared, the same 29 tests run in both builds. The only difference is that the egui build additionally compiles `theme.rs` and `ui/`, which is why `accent_comes_from_the_palette` and `every_theme_kind_maps_to_a_palette` only exist in that configuration.